



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

PROGRESS REPORT

LiDAR Point Cloud Segmentation and Object Completion on SemanticKITTI

February – May 2026

Student:

Ngo Vi Viet Anh - MSE13205

Advisor:

Dr. Doan Nhat Quang

Contents

1	Overview	3
2	Classical Segmentation Pipeline	3
2.1	Stage 1: Noise Removal	3
2.2	Stage 2: Voxel Downsampling	3
2.3	Stage 3: Ground Removal	3
2.4	Stage 4: Clustering	4
2.5	Stage 5: Geometric Filtering	4
2.6	Stage 6: Heuristic Classification	4
3	Multi-Frame Tracking	4
4	Segmentation Evaluation	5
5	Recent Progress: ShapeNet Data Preparation and PCN Training	5
5.1	ShapeNetCore v2 Exploration	5
5.2	PCN Architecture	6
5.2.1	Encoder (Stacked PointNet)	6
5.2.2	Decoder (Coarse-to-Fine)	6
5.2.3	Loss Function (Chamfer Distance)	6
5.3	Training Data Generation	7
5.4	Training Configuration	7
5.5	Training Results	7
5.5.1	Analysis	8
6	Next Steps	8

1 Overview

This report summarizes the research progress on the master thesis “LiDAR Point Cloud Segmentation and Object Completion on SemanticKITTI,” covering the period from February to May 2026.

The core research question is whether a classical (non-deep-learning) pipeline for LiDAR object segmentation can be used to extract real-world objects, which then serve as training data for a deep-learning-based point cloud completion network that bridges the synthetic-to-real domain gap.

The work is organized into two main phases:

1. **Phase 1 — Classical Segmentation Pipeline:** A 6-stage pipeline for object detection and tracking from raw LiDAR point clouds, evaluated against SemanticKITTI ground truth.
2. **Phase 2 — Deep Learning Completion:** A Point Completion Network (PCN) trained on synthetic ShapeNet data to reconstruct full 3D shapes from partial LiDAR observations.

Phase 1 has been completed and evaluated. Phase 2 has reached the training milestone, with the PCN model successfully trained on ShapeNet data.

2 Classical Segmentation Pipeline

A 6-stage pipeline was implemented to process raw LiDAR point clouds from the SemanticKITTI dataset [1] into detected and classified objects. All pipeline parameters are centralized in a single configuration dictionary (PIPELINE_CONFIG) for easy experimentation.

2.1 Stage 1: Noise Removal

Raw point clouds are loaded from .bin files as $N \times 4$ arrays (x, y, z, reflectance). Two filtering steps are applied:

- A height threshold ($z > -2.0$ m) removes points below the expected ground level.
- Open3D’s statistical outlier removal ($k = 20$ neighbors, $\sigma = 2$) eliminates isolated noise points.

2.2 Stage 2: Voxel Downsampling

The filtered point cloud is downsampled using a voxel grid with resolution 0.05 m, reducing the number of points while preserving spatial structure. This balances computational cost against the geometric detail needed for clustering.

2.3 Stage 3: Ground Removal

Ground points are separated using RANSAC plane fitting with a distance threshold of 0.2 m. A normal direction check rejects non-horizontal planes: only planes whose fitted normal has $|c|/\|[a, b, c]\| > 0.5$ are accepted as ground, preventing walls or ramps from being misclassified.

The fitted ground plane model $[a, b, c, d]$ is retained for use in the geometric filtering stage (Stage 5).

2.4 Stage 4: Clustering

Object points (non-ground) are clustered using HDBSCAN with `min_cluster_size = 10` and `min_samples = 5`. HDBSCAN was chosen over fixed-epsilon DBSCAN because LiDAR point density decreases quadratically with distance from the sensor, making a single ε value unsuitable for detecting both near and far objects.

The migration from DBSCAN to HDBSCAN was informed by a performance benchmark. Two Python implementations were evaluated on SemanticKITTI sequence 00, frame 0 (~45k object points):

Table 1: HDBSCAN implementation comparison

Implementation	Time (s)	Clusters
Open3D DBSCAN (C++, $\varepsilon = 0.5$)	~0.01	—
hdbscan package (0.8.42)	0.48	293
sklearn.cluster.HDBSCAN (1.8.0)	3.72	309

The dedicated `hdbscan` package was selected as it is $7.7\times$ faster than scikit-learn’s implementation while producing comparable cluster counts.

2.5 Stage 5: Geometric Filtering

Each cluster is enclosed in an oriented bounding box (OBB). Clusters are filtered based on geometric criteria:

- Minimum bounding box volume: 0.5 m^3 .
- Maximum center height above the ground plane: 3.0 m. The height is computed as the signed distance from the bbox center to the RANSAC-fitted ground plane:

$$h = \frac{a \cdot c_x + b \cdot c_y + c \cdot c_z + d}{\sqrt{a^2 + b^2 + c^2}} \quad (1)$$

where (c_x, c_y, c_z) is the bbox center and $[a, b, c, d]$ is the ground plane model. This ground-relative measurement is robust to slopes and sensor tilt, unlike a raw sensor-frame z -threshold.

2.6 Stage 6: Heuristic Classification

Detected objects are classified based on bounding box dimensions into five categories: pedestrian, car, truck, traffic sign, and vegetation. While simple, this provides a baseline classification that can be replaced with a learned classifier in future work.

3 Multi-Frame Tracking

A `CentroidTracker` assigns persistent IDs to detected objects across frames using the Hungarian algorithm for centroid-based assignment. Key parameters:

- Maximum match distance: 2.0 m — centroids farther apart are not matched.
- Maximum disappeared frames: 5 — tracks are dropped after 5 consecutive frames without a match.

Point clouds are transformed to a global coordinate frame using:

$$T_{\text{global}} = T_{\text{pose}}^{(i)} \cdot T_r \quad (2)$$

where $T_{\text{pose}}^{(i)}$ is the ego-motion pose (camera frame) and T_r is the LiDAR-to-camera calibration matrix from `calib.txt`.

4 Segmentation Evaluation

An evaluation harness (`evaluate.py`) compares pipeline detections against SemanticKITTI ground-truth instance labels. For each frame, detected clusters and GT instances are matched using IoU (Intersection over Union), with a threshold of 0.3. The metrics computed are:

- **Precision** — fraction of detected clusters that match a GT instance.
- **Recall** — fraction of GT instances matched by a detection.
- **F1** — harmonic mean of precision and recall.
- **Mean IoU** — average IoU of matched pairs.

GT instances are derived from SemanticKITTI “thing” classes (cars, pedestrians, cyclists, etc.). The baseline results on sequence 00 (20 frames, IoU threshold ≥ 0.3):

Table 2: Segmentation evaluation: DBSCAN vs. HDBSCAN baseline

Method	Precision	Recall	F1
DBSCAN ($\varepsilon = 0.5$)	0.188	0.804	0.305
HDBSCAN	0.154	0.868	0.261

HDBSCAN improves recall by 6.4 percentage points but reduces precision, resulting in a lower F1. The higher recall reflects HDBSCAN’s ability to detect distant objects that fixed-epsilon DBSCAN splits into multiple fragments. The lower precision indicates more false positives, which can be addressed by tightening geometric filters.

5 Recent Progress: ShapeNet Data Preparation and PCN Training

Recent Progress (April 25 – May 6, 2026)

The following sections describe work completed between April 25 and May 6, 2026, representing the transition from classical segmentation (Phase 1) to deep-learning-based completion (Phase 2).

5.1 ShapeNetCore v2 Exploration

ShapeNetCore v2 [2] was selected as the synthetic training data source for point cloud completion. Three object categories relevant to the KITTI driving domain [3] were identified:

Table 3: ShapeNet categories used for completion training

Category	Synset ID	Models
Car	02958343	3,533
Bus	02924116	939
Motorcycle	03790512	337
Total		4,809

Data was accessed via the HuggingFace API (`huggingface_hub`). Mesh models (OBJ format) were loaded and visualized using `trimesh`. A preliminary comparison between complete point clouds (sampled from mesh surfaces) and simulated partial LiDAR scans was conducted in the data exploration notebook to validate the completion task setup.

5.2 PCN Architecture

The Point Completion Network (PCN) [4] was implemented from scratch in PyTorch. The architecture consists of three components:

5.2.1 Encoder (Stacked PointNet)

The encoder uses a two-stage PointNet design to extract a global feature vector from an unordered input point cloud of 2,048 points:

- **Stage 1:** Conv1d layers ($3 \rightarrow 128 \rightarrow 256$) extract per-point local features. A max-pooled global feature is concatenated back to each point.
- **Stage 2:** Conv1d layers ($512 \rightarrow 512 \rightarrow 1024$) process the enriched features. A final max pool produces the global feature vector $\mathbf{g} \in \mathbb{R}^{1024}$.

Total parameters: 6.87M.

5.2.2 Decoder (Coarse-to-Fine)

The decoder produces the completed point cloud in two stages:

1. **Coarse output:** Fully-connected layers ($1024 \rightarrow 1024 \rightarrow 1024 \rightarrow 3072$) produce 1,024 coarse points that approximate the overall shape.
2. **Fine output:** Each coarse point is combined with a 4×4 2D grid (16 points). A folding MLP ($1029 \rightarrow 512 \rightarrow 512 \rightarrow 3$) maps each grid point to a 3D offset, producing $1,024 \times 16 = 16,384$ fine points that capture surface detail.

5.2.3 Loss Function (Chamfer Distance)

The training loss combines Chamfer Distance (CD) on both output resolutions:

$$\mathcal{L} = \text{CD}(\hat{Y}_{\text{coarse}}, Y) + \alpha \cdot \text{CD}(\hat{Y}_{\text{fine}}, Y), \quad \alpha = 0.5 \quad (3)$$

where the Chamfer Distance between predicted set P and ground truth set Q is:

$$\text{CD}(P, Q) = \frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|_2^2 + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|_2^2 \quad (4)$$

A chunked implementation (chunk size 2,048) computes distance matrices in segments to avoid GPU memory overflow when processing 16,384-point sets. The fine CD is computed on a random subsample of 4,096 points from both prediction and ground truth for efficiency.

5.3 Training Data Generation

The `ShapeNetCompletionDataset` loader generates training pairs on-the-fly:

- **Ground truth (dense):** 16,384 points sampled uniformly from the mesh surface using `trimesh`.
- **Input (partial):** Simulated LiDAR scans via `Open3D RaycastingScene` depth rendering (256×256 resolution) from randomized viewpoints (elevation $[-20^\circ, 30^\circ]$, azimuth $[0^\circ, 360^\circ]$, radius $[2.5, 4.5]$ m). Depth images are back-projected to 3D and subsampled to 2,048 points.
- **Real-world scaling:** ShapeNet models are normalized; they are rescaled to match real-world dimensions (car = 4.5 m, bus = 10 m, motorcycle = 2.2 m).

Total training samples: 3,834.

5.4 Training Configuration

Table 4: PCN training hyperparameters

Parameter	Value
Optimizer	Adam
Initial learning rate	10^{-4}
LR scheduler	StepLR ($\times 0.5$ every 40 epochs)
Batch size	8
Epochs	100
GPU	NVIDIA RTX 3070 Ti (CUDA)
Total training time	~ 18 hours

5.5 Training Results

Training was completed for all 100 epochs. Table 5 presents the key metrics at selected checkpoints.

Table 5: PCN training results on ShapeNetCore v2

Epoch	Train Loss	Val Loss	CD Coarse	CD Fine	F-Score	LR
1	0.672	0.586	0.396	0.380	—	10^{-4}
10	0.463	0.448	0.317	0.261	0.748	10^{-4}
40	0.403	0.400	0.287	0.225	0.809	5×10^{-5}
58	0.384	0.376	0.272	0.210	—	5×10^{-5}
80	0.380	0.374	0.270	0.209	0.839	2.5×10^{-5}
100	0.371	0.375	0.270	0.210	0.841	2.5×10^{-5}

5.5.1 Analysis

- **Convergence:** The loss decreased rapidly during the first 40 epochs and converged around epoch 60–70. The final 30 epochs produced minimal improvement, with the validation loss plateauing at approximately 0.374.
- **Learning rate schedule:** Each LR reduction (at epochs 40 and 80) produced a small but measurable improvement, confirming the effectiveness of the step decay strategy.
- **F-Score:** The final F-Score of 0.841 indicates good shape reconstruction quality on synthetic ShapeNet data. However, this metric reflects performance on clean synthetic objects and does not yet account for the domain gap with real LiDAR data.
- **Best checkpoint:** Epoch 80 achieved the lowest validation loss (0.374) and an F-Score of 0.839. The epoch 100 checkpoint shows a marginally higher validation loss (0.375), indicating no overfitting but also no further improvement.
- **Overfitting:** The gap between training and validation loss remained stable throughout, with no divergence observed. This suggests the model generalizes well within the ShapeNet domain.

6 Next Steps

1. **Qualitative evaluation of PCN.** Visualize completion results on ShapeNet test samples (input partial → coarse output → fine output) to inspect shape quality and identify failure modes.
2. **Evaluate PCN on real KITTI objects.** Run the trained PCN on partial point clouds extracted by the segmentation pipeline (Step 7 integration). This is the critical domain-transfer test: the model was trained on clean synthetic data and must generalize to noisy, sparse LiDAR observations.
3. **Add Earth Mover’s Distance (EMD) loss.** Supplement the Chamfer Distance with an EMD term (Sinkhorn approximation) for the coarse output to improve point distribution uniformity.
4. **Domain adaptation.** Bridge the ShapeNet-to-KITTI domain gap by applying the `simulate_lidar_noise()` augmentation during fine-tuning on real data extracted by the pipeline.
5. **Tune segmentation filters.** Tighten geometric filters to reduce false positives from HDBSCAN clustering and improve segmentation precision without sacrificing the recall gains.
6. **Explore stronger completion architectures.** If PCN quality proves insufficient for the thesis, evaluate PoinTr or SeedFormer as alternatives with potentially better reconstruction fidelity.

References

- [1] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall, “SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences,” in *Proc. of the IEEE/CVF International Conf. on Computer Vision (ICCV)*, 2019.
- [2] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, “ShapeNet: An Information-Rich 3D Model Repository,” Stanford University — Princeton University — Toyota Technological Institute at Chicago, Tech. Rep. arXiv:1512.03012 [cs.GR], 2015.
- [3] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361.
- [4] W. Yuan, T. Khot, D. Held, C. Mertz, and M. Hebert, “Pcn: Point completion network,” in *3D Vision (3DV), 2018 International Conference on*, 2018.